

Name:Ravi Shankar Tripathi

AWS Account ID: 797767715969

Region: ap-south-1 (Mumbai)

Task 1: Deploy Flask Backend and Express Frontend on a Single EC2 Instance

objective:To launch a single Amazon EC2 virtual machine and configure both the Flask REST API backend and Express frontend web application to run on the same server.

Implementation Steps :

1.Launched an AWS EC2 instance using the Ubuntu Server (t2.micro) AMI.

2.Configured the Security Group inbound rules to allow:

Port 22: For remote SSH terminal access.

Port 5000: For the Flask backend API.

Port 3000: For the Express frontend client.

3.Connected to the instance via SSH and updated system packages:

Terminal commands:

```
sudo apt update -y
```

```
sudo apt install -y python3 python3-pip python3-venv nodejs npm git
```

Cloned the application source code onto the server

```
sudo apt install -y nodejs
```

Set up the backend service:

- Created a Python virtual environment and installed dependencies from `requirements.txt`.
- Started the Flask backend service bound to `0.0.0.0:5000`.

Set up the frontend service:

- Navigated to the frontend directory and executed `npm install`.
- Set the backend connection URL to localhost
(`[http://127.0.0.1:5000](http://127.0.0.1:5000)`).
- Started the Express frontend server listening on Port 3000

```
Windows PowerShell
npm notice
npm notice New major version of npm available! 10.8.2 -> 12.1.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v12.1.0
npm notice To update run: npm install -g npm@12.1.0
npm notice
ubuntu@ip-172-31-2-246:~/app/frontend$ pm2 start server.js --name "express-frontend" --env BACKEND_URL="http://localhost:5000"
[PM2] Starting /home/ubuntu/app/frontend/server.js in fork_mode (1 instance)
[PM2] Done.

table
  id   name           mode   up   status   cpu   memory
  --   -
  1    express-frontend fork   0    online  0%    22.1mb
  0    flask-backend  fork   0    online  0%    35.3mb

host metrics | cpu: 9.7% | ram usage: 39.3% | enX0: 0.103mb/s 0.005mb/s
ubuntu@ip-172-31-2-246:~/app/frontend$ cd ..
ubuntu@ip-172-31-2-246:~/app$ pm2 status

table
  id   name           mode   up   status   cpu   memory
  --   -
  1    express-frontend fork   0    online  0%    57.6mb
  0    flask-backend  fork   0    online  0%    35.3mb

host metrics | cpu: 0.2% | ram usage: 39.2%
ubuntu@ip-172-31-2-246:~/app$ cd frontend
ubuntu@ip-172-31-2-246:~/app/frontend$ pm2 restart express-frontend --update-env --env BACKEND_URL="http://127.0.0.1:5000"
[PM2][ERROR] Using --env [env] without passing the ecosystem.config.js does not work
ubuntu@ip-172-31-2-246:~/app/frontend$ pm2 delete express-frontend
[PM2] Applying action deleteProcessId on app [express-frontend](ids: [ 1 ])
[PM2] [express-frontend](1) ✓

table
  id   name           mode   up   status   cpu   memory
  --   -
  0    flask-backend  fork   0    online  0%    35.3mb

host metrics | cpu: 0.2% | ram usage: 42%
ubuntu@ip-172-31-2-246:~/app/frontend$ BACKEND_URL="http://127.0.0.1:5000" pm2 start server.js --name "express-frontend"
[PM2] Starting /home/ubuntu/app/frontend/server.js in fork_mode (1 instance)
[PM2] Done.

table
  id   name           mode   up   status   cpu   memory
  --   -
  0    flask-backend  fork   0    online  0%    35.3mb
```

Browser tabs: Tutorials, docs.google, Assignments, GitHub, Online, WhatsApp, Sri Krish, Python, New Tail, How to, Free Cls, Launch, Inst, AWS Co.

Address bar: ap-south-1.console.aws.amazon.com/ec2/home?region=ap-south-1#InstanceDetails:instanceId=i-09838226e690beb49

Navigation: EC2 > Instances > i-09838226e690beb49

EC2

- Dashboard
- AWS Global View
- Events
- Instances
 - Instances
 - Instance Types
 - Launch Templates
 - Spot Requests
 - Savings Plans
 - Reserved Instances
 - Dedicated Hosts
 - Capacity Reservations
 - Capacity Manager
- Images
 - AMIs
 - AMI Catalog
- Elastic Block Store

Updated less than a minute ago

Instance ID
i-09838226e690beb49

IPv6 address
-

Hostname type
IP name: ip-172-31-2-246.ap-south-1.compute.internal

Answer private resource DNS name
IPv4 (A)

Auto-assigned IP address
13.127.43.55 [Public IP]

IAM role
-

IMDSv2
Required

Public IPv4 address
13.127.43.55 | open address

Instance state
Running

Private IP DNS name (IPv4 only)
ip-172-31-2-246.ap-south-1.compute.internal

Instance type
t2.micro

VPC ID
vpc-0ebd6089bdf6a028c

Subnet ID
subnet-0266637b844e32155

Instance ARN
arn:aws:ec2:ap-south-1:797767715969:instance/i-09838226e690beb49

Private IPv4 addresses
172.31.2.246

Public DNS
ec2-13-127-43-55.ap-south-1.compute.amazonaws.com | open address

Elastic IP addresses
-

AWS Compute Optimizer finding
Opt-in to AWS Compute Optimizer for recommendation

Auto Scaling Group name
-

Managed
false

Footer: © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Task 2: Deploy Flask Backend and Express Frontend on Separate EC2 Instances

Implementation Steps:

1. Launched two separate Ubuntu EC2 instances in the default VPC:

Instance A (name:course): Dedicated to the Flask API.

Instance B (Frontend): Dedicated to the Express Web UI.

2. Configured custom Security Group rules for both instances:

Backend Security Group: Allowed inbound TCP traffic on Port 5000 from the Frontend instance's IP / Security Group.

Frontend Security Group: Allowed inbound TCP traffic on Port 3000 from 0.0.0.0/0 (public access).

3. Connected to **Instance A**, installed Python dependencies, and started the Flask application listening on Port 5000.

4. Connected to **Instance B**, installed Node.js dependencies, and configured the frontend environment variable:

5. Started the Express frontend server on Port 3000.

6. Tested form submission in the browser to confirm that data entered into Instance B successfully reaches the API on Instance A.

ap-south-1.console.aws.amazon.com/ec2/home?region=ap-south-1#instances:instanceState=running

EC2 > Instances

Instances (2) Info

Saved filter sets
Choose filter set
Filter Instance by attribute or tag (case-sensitive)

Instance state = running Clear filters

☐	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	F
☐	course	i-09838226e690beb49	Running	t2.micro	2/2 checks passed	ap-south-1b	ec2-13-127-43-55.ap-s...	1
☐	Frontend-Expr...	i-0dc7878981d4631c7	Running	t2.micro	Initializing	ap-south-1b	ec2-15-252-142-228.ap...	1

All instances

Instance alarm states

In alarm
None Show instances in alarm

[View in CloudWatch](#)

Service health

EC2 service status
Operating normally

[View Service Health Dashboard](#)

Health events affecting EC2 instances

Open events
None Show instances

Upcoming events
None Show instances

[View AWS Health Dashboard](#)

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

25°C Mostly cloudy 20:19 24-09-2026

ap-south-1.console.aws.amazon.com/ec2/home?region=ap-south-1#SecurityGroup:group-id=sg-0f705423afe44781e

EC2 > Security Groups > sg-0f705423afe44781e

sg-0f705423afe44781e - launch-wizard-1

Inbound security group rules successfully modified on security group (sg-0f705423afe44781e | launch-wizard-1)

Details

Security group name launch-wizard-1	Security group ID sg-0f705423afe44781e	Description launch-wizard-1 created 2026-09-24T13:00:31.356Z	VPC ID vpc-0ebd6089bdf6a028c
Owner 797767715969	Inbound rules count 3 Permission entries	Outbound rules count 1 Permission entry	

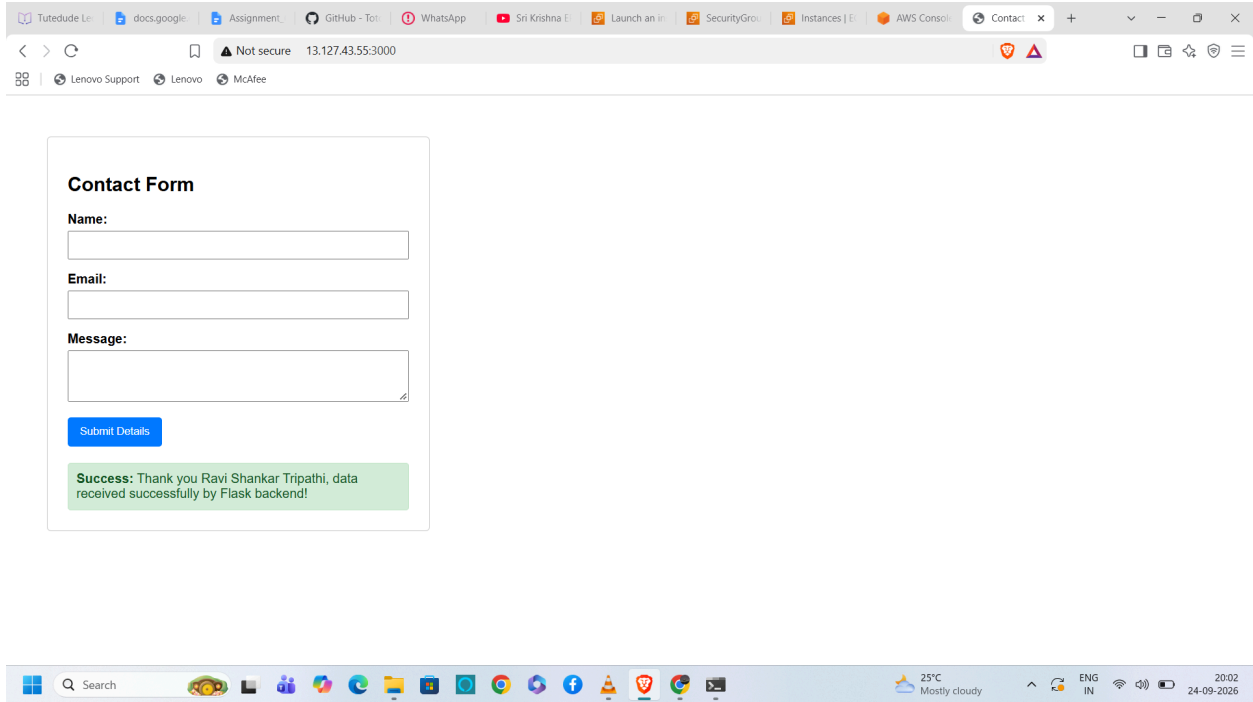
Inbound rules Outbound rules Sharing VPC associations Related resources - new Tags

Inbound rules (3)

☐	Name	Security group rule ID	IP version	Type	Protocol	Port range
☐	-	sgr-05c564b9c2bd246ba	IPv4	Custom TCP	TCP	3000
☐	-	sgr-08250769a34924719	IPv4	Custom TCP	TCP	5000

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

25°C Mostly cloudy 19:46 24-09-2026



Task 3: Deploy Flask Backend and Express Frontend as Docker Containers using AWS ECR, ECS, and VPC

objective:

To containerize both tiers of the application using Docker, push the container images to Amazon Elastic Container Registry (ECR), and orchestrate deployment using serverless Amazon Elastic Container Service (AWS ECS Fargate).

Implementation Steps:

1.Authentication and ECR Setup

Logged in local Docker CLI to the AWS ECR registry:

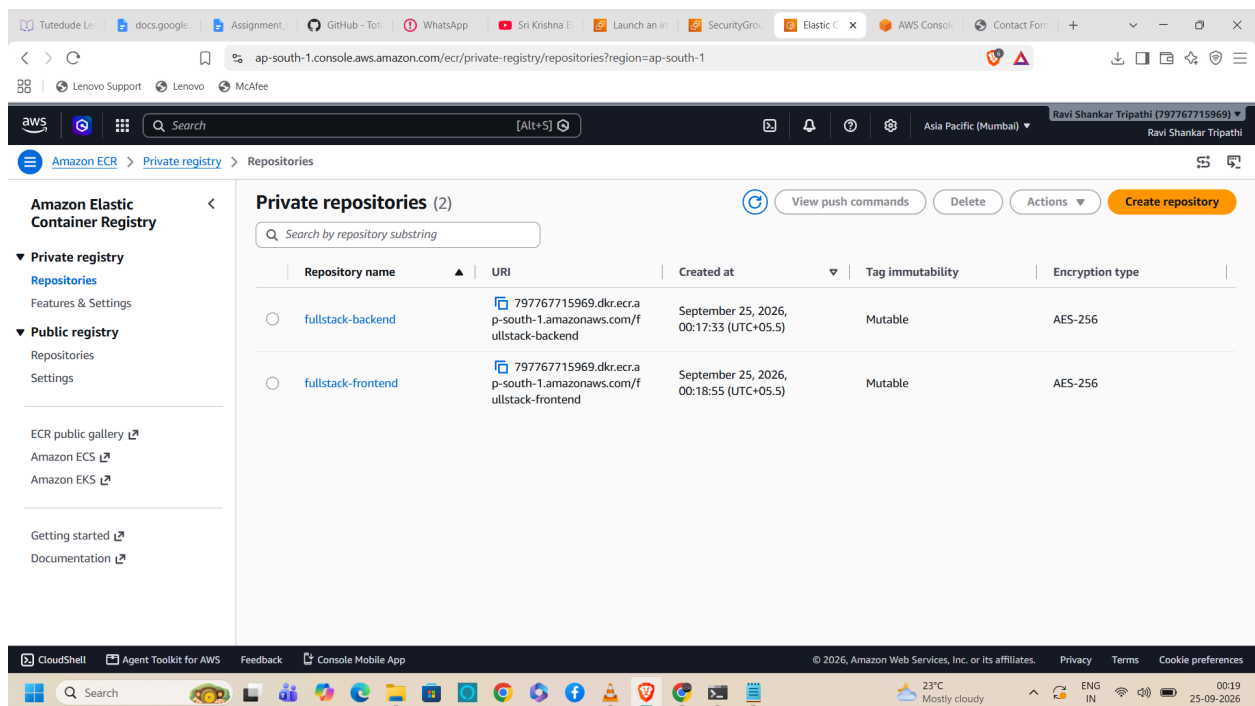
Command:

```
aws ecr get-login-password --region ap-south-1 | docker  
login --username AWS --password-stdin  
797767715969.dkr.ecr.ap-south-1.amazonaws.com
```

Created two private ECR repositories:

fullstack-backend

fullstack-frontend



2. Building and Pushing Docker Images

Terminal commands:

```
# Backend Build and Push
```

```
cd app/backend
```

```
docker build -t  
797767715969.dkr.ecr.ap-south-1.amazonaws.com/fullstack  
-backend:latest .
```

```
docker push  
797767715969.dkr.ecr.ap-south-1.amazonaws.com/fullstack  
-backend:latest
```

Frontend Build and Push

```
cd ../frontend
```

```
docker build -t  
797767715969.dkr.ecr.ap-south-1.amazonaws.com/fullstack  
-frontend:latest .
```

```
docker push  
797767715969.dkr.ecr.ap-south-1.amazonaws.com/fullstack  
-frontend:latest
```

Browser tabs: TuteDude Le, docs.google, Assignment, GitHub - Tot, WhatsApp, Sri Krishna E, Launch an in, SecurityGro, Elastic C, AWS Consol, Contact For

Address bar: ap-south-1.console.aws.amazon.com/ecr/repositories/private/797767715969/fullstack-backend/_/details?region=ap-south-1

Taskbar: Lenovo Support, Lenovo, McAfee

Navigation: Amazon ECR > Private registry > Repositories > fullstack-backend

Amazon Elastic Container Registry

- ▼ Private registry
 - Repositories
 - fullstack-backend**
 - Features & Settings
- ▼ Public registry
 - Repositories
 - Settings

ECR public gallery [↗](#)
Amazon ECS [↗](#)
Amazon EKS [↗](#)

Getting started [↗](#)
Documentation [↗](#)

fullstack-backend

Summary | **Images** | Lifecycle policy | Permissions | Repository tags

Images (1) [Info](#) [Delete](#) [Copy URI](#) [Details](#) [Scan](#) [View push commands](#)

☐	Image tags	Type	Created at	Image size (MB)	Image digest	Last pulled at
☐	latest	Image	September 25, 2026, 00:32:21 (UTC+05.5)	50.05	sha256:c294aad...	-

CloudShell | Agent Toolkit for AWS | Feedback | Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

23°C Mostly cloudy 00:36 25-09-2026

Browser tabs: TuteDude Le, docs.google, Assignment, GitHub - Tot, WhatsApp, Sri Krishna E, Launch an in, SecurityGro, Elastic C, AWS Consol, Contact For

Address bar: ap-south-1.console.aws.amazon.com/ecr/repositories/private/797767715969/fullstack-frontend/_/details?region=ap-south-1

Taskbar: Lenovo Support, Lenovo, McAfee

Navigation: Amazon ECR > Private registry > Repositories > fullstack-frontend

Amazon Elastic Container Registry

- ▼ Private registry
 - Repositories
 - fullstack-frontend**
 - Features & Settings
- ▼ Public registry
 - Repositories
 - Settings

ECR public gallery [↗](#)
Amazon ECS [↗](#)
Amazon EKS [↗](#)

Getting started [↗](#)
Documentation [↗](#)

fullstack-frontend

Summary | **Images** | Lifecycle policy | Permissions | Repository tags

Images (1) [Info](#) [Delete](#) [Copy URI](#) [Details](#) [Scan](#) [View push commands](#)

☐	Image tags	Type	Created at	Image size (MB)	Image digest	Last pulled at
☐	latest	Image	September 25, 2026, 00:39:57 (UTC+05.5)	49.06	sha256:5df9705...	-

CloudShell | Agent Toolkit for AWS | Feedback | Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

23°C Mostly cloudy 00:40 25-09-2026

3. IAM Role Configuration & ECS Cluster Creation

Resolved the missing service-linked role by executing:

command:

```
aws iam create-service-linked-role --aws-service-name  
ecs.amazonaws.com
```

Created the task execution role (ecsTaskExecutionRole) and attached the managed execution policy to permit ECR image pulls:

Command:

```
aws iam create-role --role-name ecsTaskExecutionRole  
--assume-role-policy-document  
'{"Version":"2012-10-17","Statement":[{"Effect":"Allow","Principal":{"Service":"ecs-tasks.amazonaws.com"},"Action":"sts:AssumeRole"}]}'
```

```
aws iam attach-role-policy --role-name ecsTaskExecutionRole  
--policy-arn  
arn:aws:iam::aws:policy/service-role/AmazonECSTaskExecutionRolePolicy
```

Created the ECS cluster named **course** configured for **AWS Fargate (Serverless)**.

4. Task Definition Registration

Defined a multi-container task definition under family fullstack-task:

Compute Allocation: CPU: 1 vCPU, Memory: 2 gb, Network mode: awsvpc

Container 1 (backend):

Image:

797767715969.dkr.ecr.ap-south-1.amazonaws.com/fullstack-backend:latest

Port: 5000(TCP)

Container 2 (frontend):

Image:

797767715969.dkr.ecr.ap-south-1.amazonaws.com/fullstack-frontend:latest

Port: 3000(TCP)

Environment Variable:

BACKEND_URL=http://127.0.0.1:5000

Task definition successfully created
fullstack-task:1 has been successfully created. You can use this task definition to deploy a service or run a task.

fullstack-task:1

Overview

ARN arn:aws:ecs:ap-south-1:797767715:969:task-definition/fullstack-task:1	Status ACTIVE	Time created September 25, 2026, 02:49 (UTC+5:30)	App environment Fargate
Task role -	Task execution role ecsTaskExecutionRole	Operating system/Architecture Linux/X86_64	Network mode awsvpc
Fault injection Turned off			

Containers

Task size

Task CPU

Task memory

Task definition successfully created
fullstack-task:2 has been successfully created. You can use this task definition to deploy a service or run a task.

fullstack-task:2

Overview

ARN arn:aws:ecs:ap-south-1:797767715:969:task-definition/fullstack-task:2	Status ACTIVE	Time created September 25, 2026, 03:02 (UTC+5:30)	App environment Fargate
Task role -	Task execution role ecsTaskExecutionRole	Operating system/Architecture -	Network mode awsvpc
Fault injection -			

Containers

Task size

Task CPU
512 units (0.5 vCPU)

Task memory
1,024 MiB (1 GiB)

5. Task Launch and Live Verification

Executed the task inside the course cluster using the Fargate launch type.

Configured networking using default public subnets with **Public IP: Enabled**.

Monitored the task until it transitioned to the **Running** state with assigned Public IP: **43.204.19.83**.

Accessed the live endpoint in the browser at http://43.204.19.83:3000.

Verified form functionality: input data submitted on the frontend successfully reached the backend over the internal loopback adapter (127.0.0.1:5000).

